

作业：CI/CD 配置

作业目标

为项目配置持续集成 (CI) 流水线, 实现代码提交后自动运行测试与 Lint 检查, 并在 README 中展示运行状态。

workflow 配置（团队共同完成）

一人负责创建, 其余成员通过 PR Review 参与, 共同保证 workflow 正确运行。

基础 CI 工作流

`.github/workflows/ci.yml` — 后端与前端测试并行运行：

```
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - run: pip install -r backend/requirements.txt pytest coverage ruff
      - run: ruff check backend/
      - run: coverage run -m pytest backend/tests && coverage xml
      - uses: codecov/codecov-action@v4
        with:
          token: ${ secrets.CODECOV_TOKEN }
          files: coverage.xml
          flags: backend

  frontend:
    runs-on: ubuntu-latest
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'npm'
  - run: npm ci --prefix frontend
  - run: npm run lint --prefix frontend
  - run: npm test --prefix frontend -- --coverage --ci
  - uses: codecov/codecov-action@v4
    with:
      token: ${ secrets.CODECOV_TOKEN }
      files: frontend/coverage/lcov.info
      flags: frontend

```

后端成员任务

1. 确保测试在 CI 环境可运行

CI 环境没有本地数据库，需要用环境变量或 SQLite 替代生产配置，并通过 `pyproject.toml` 明确测试路径：

```

# pyproject.toml
[tool.pytest.ini_options]
testpaths = ["tests"]
# 需要数据库时，CI 用 SQLite in-memory:
# env = ["DATABASE_URL=sqlite:///memory:"]

[tool.coverage.run]
source = ["."]
omit = ["tests/*", "venv/*", "migrations/*"]

```

如果项目依赖 PostgreSQL，可在工作流中启动服务容器：

```

backend:
  services:
    postgres:
      image: postgres:16
      env:
        POSTGRES_DB: testdb
        POSTGRES_USER: user
        POSTGRES_PASSWORD: pass

```

```

ports: ['5432:5432']
options: >-
  --health-cmd pg_isready
  --health-interval 5s
  --health-retries 5
env:
  DATABASE_URL: postgresql://user:pass@localhost:5432/testdb

```

2. 通过 Lint 检查

本地先运行，确保零警告后再提交：

```

pip install ruff
ruff check .          # 查看所有问题
ruff check . --fix    # 自动修复可修复的问题

```

常见需要手动处理的问题：

ruff 错误码	含义	典型修复
E501	行过长	换行或配置 <code>line-length = 120</code>
F401	未使用的导入	删除多余 import
F841	赋值后未使用的变量	改为 <code>_</code> 或删除
E711	<code>== None</code> 比较	改为 <code>is None</code>

3. 覆盖率报告本地验证

提交前在本地确认覆盖率，避免 CI 上传失败：

```

coverage run -m pytest tests/
coverage report          # 终端查看
coverage html           # 生成 htmlcov/index.html 在浏览器查看

```

4. 要求

要求	说明
pytest 全部通过	CI 中无失败用例，包含 CI 环境特有的环境变量设置
ruff 零警告	Lint job 显示绿色
核心覆盖率	> 60% (沿用上次作业的测试，新增的接口也需覆盖)

前端成员任务

1. 配置测试命令支持 CI 模式

CI 终端是非交互式的，Jest 默认会进入 watch 模式导致卡死，需要显式指定 `--ci` 标志：

```
// package.json (Jest)
{
  "scripts": {
    "test": "jest --ci --coverage",
    "lint": "eslint src/ --max-warnings 0"
  },
  "jest": {
    "collectCoverageFrom": ["src/**/*.{js,jsx,ts,tsx}", "!src/index.{js,jsx}"]
  }
}
```

```
// package.json (Vitest)
{
  "scripts": {
    "test": "vitest run --coverage",
    "lint": "eslint src/ --max-warnings 0"
  }
}
```

2. 处理常见 CI 环境差异

前端测试在 CI 中容易因环境缺失而失败，常见问题及解法：

问题	原因	解法
window is not defined	CI 无浏览器环境	Jest 配置 <code>testEnvironment: 'jsdom'</code>
快照测试失败	本地与 CI 渲染结果不一致	提交前运行 <code>jest -u</code> 更新快照
环境变量未定义	<code>.env</code> 不提交到仓库	在工作流中用 <code>env:</code> 注入测试用变量
ESLint 本地通过 CI 失败	版本或配置不一致	锁定 eslint 版本，统一 <code>.eslintrc</code>

注入测试环境变量示例：

```
frontend:
  env:
    VITE_API_URL: http://localhost:3000 # 测试时使用的 mock 地址
```

3. 本地验证

提交前完整运行一遍，保证与 CI 结果一致：

```
npm run lint      # ESLint 检查，应零错误
npm test          # 全部测试通过，生成 coverage/
```

4. 要求

要求	说明
npm test 全部通过	CI 中无失败用例，无快照不一致
ESLint 零错误	使用 --max-warnings 0，Lint job 显示绿色
覆盖率报告生成	frontend/coverage/lcov.info 可被 Codecov 上传

状态徽章

CI 运行成功后，在 README.md 顶部添加徽章：

```
[![CI](https://github.com/<用户名>/<仓库名>/actions/workflows/ci.yml/badge.svg)]
(https://github.com/<用户名>/<仓库名>/actions)
[![Backend Coverage](https://codecov.io/gh/<用户名>/<仓库名>/branch/main/graph/
badge.svg?flag=backend)](https://codecov.io/gh/<用户名>/<仓库名>)
[![Frontend Coverage](https://codecov.io/gh/<用户名>/<仓库名>/branch/main/graph/
badge.svg?flag=frontend)](https://codecov.io/gh/<用户名>/<仓库名>)
```

目录结构

```
project/
├── .github/
│   └── workflows/
│       └── ci.yml          # CI 工作流
├── backend/
│   └── tests/              # 上次作业的测试代码
├── frontend/
│   └── src/
│       └── __tests__/      # 上次作业的测试代码
```

```
| docs/
|   | contributions/
|   |   | 09-cicd/
|   |   |   | zhangsan.md
|   |   |   | lisi.md
|   |   |   |
|   |   |   | README.md # 添加状态徽章
```

提交内容

Git 仓库

1. `.github/workflows/ci.yml` 工作流文件（通过 PR 合并）
2. 更新 `README.md`，添加 CI 及覆盖率状态徽章
3. `docs/contributions/09-cicd/你的名字.md`

学习通截图

1. Git 提交记录 — `git log --author="YOUR_NAME" --oneline` 截图
2. CI 运行成功截图 — GitHub Actions 页面，backend 和 frontend 两个 job 均显示绿色 ✓
3. README 徽章截图 — 显示 CI 及覆盖率徽章

💡 工作流文件合并到 main 后自动触发，确保两个 job 都通过后再截图。

截止时间：下次课前

个人贡献说明模板

文件路径：`docs/contributions/09-cicd/你的名字.md`

CI/CD 配置贡献说明

姓名：XXX 学号：XXX 角色：前端 / 后端 日期：XXXX-XX-XX

完成的工作

工作流相关

- [] 参与编写 / 审查 `.github/workflows/ci.yml`
- [] 配置 Codecov 覆盖率上传 (backend / frontend flag)
- [] 添加 README 状态徽章

代码适配

- [] 本地测试命令与 CI 一致，无需额外配置

- [] 代码通过 Lint 检查 (ruff / ESLint)
- [] 核心覆盖率达标 (> 60%)

可选项

- [] 配置 Dependabot 自动更新依赖
- [] 集成 CodeRabbit AI 代码审查
- [] 使用 act 本地验证工作流

PR 链接

- PR #X: <https://github.com/xxx/xxx/pull/X>

CI 运行链接

- <https://github.com/xxx/xxx/actions/runs/XXX>

遇到的问题和解决

1. 问题: XXX → 解决: XXX

心得体会
(简要说明收获)

评分标准

项目	分值	说明
CI 工作流配置	30%	YAML 正确, 包含 backend 和 frontend 两个 job
CI 运行成功	30%	GitHub Actions 全部 job 显示绿色
覆盖率上传	15%	Codecov 可查到后端/前端覆盖率
README 徽章	10%	徽章正确显示 CI 状态
个人贡献说明	15%	内容清晰, 有截图佐证

额外加分项

- 配置 Dependabot 自动更新依赖
- 集成 CodeRabbit 实现 AI 代码自动审查
- 使用 act 本地验证工作流并在贡献说明中记录过程
- 添加构建矩阵 (多 Python 版本或多 Node 版本)